

ZARC DRT Derivation and Peak-Parameter Mapping

Souradeep Mitra

2026-07-21

1 Purpose

This note collects the mathematical discussion used in the MATLAB single-file DRT workflow:

1. the exact DRT of a ZARC element,
2. the resistance, time constant, and capacitance-style quantities extracted from a refined DRT peak,
3. the full-width-at-half-maximum (FWHM) relation that can be used to estimate the ZARC exponent n from peak width.

The implementation context is the single-file workflow `Impedance_analysis_singlefile.m`.

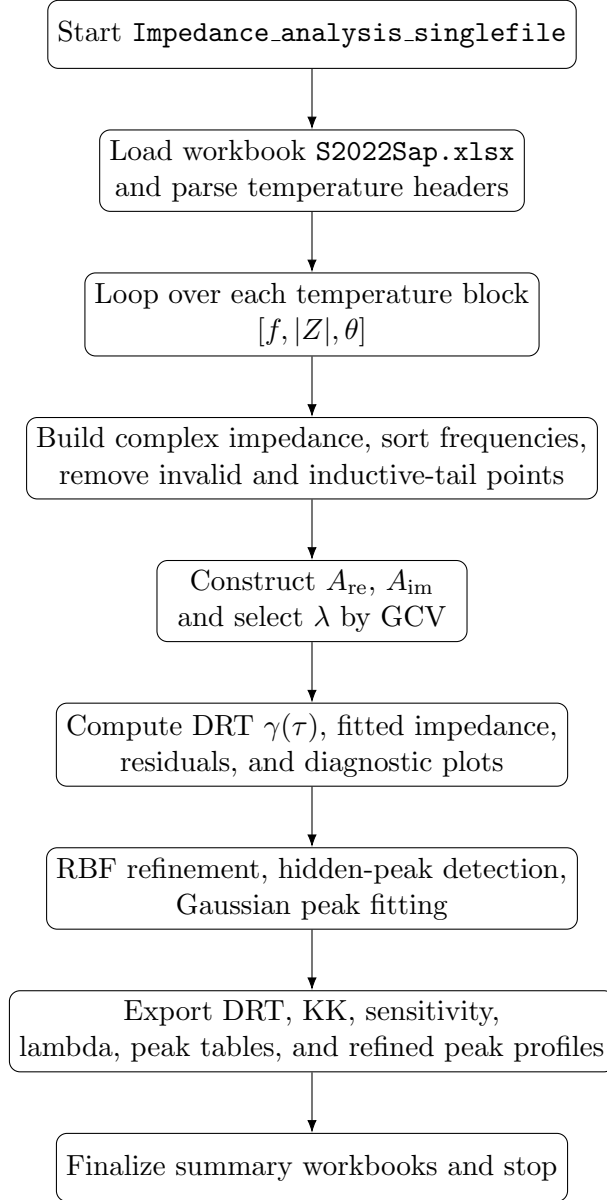
2 Code-Level Workflow Summary

At a high level, the MATLAB workflow executes the following steps for the default workbook `S2022Sap.xlsx`:

1. load the multi-temperature workbook and split it into three-column blocks,
2. convert magnitude-phase data into complex impedance,
3. remove invalid frequencies and the high-frequency inductive tail,
4. build the DRT projection matrices and select the regularization parameter by generalized cross-validation,
5. recover the DRT vector and reconstruct the fitted impedance,
6. refine the DRT peak structure using an RBF interpolation plus Gaussian basis fit,
7. convert refined peaks into exported R , C , and heuristic n values,
8. run Kramers-Kronig diagnostics,
9. write temperature-wise Excel outputs for the DRT, peak table, KK metrics, and refined peak profiles.

The code therefore has two complementary layers: a mathematical inversion layer, and a reporting layer that exports all derived quantities for post-processing.

3 Execution Flowchart



4 How the MATLAB Functions Map to the Mathematics

The single-file implementation is structured as a top-level orchestration function plus a set of local helper functions.

Input and Preprocessing Helpers

- `load_input_dataset_2011`: reads the numeric workbook block and extracts temperature labels from the first header row.
- `load_f_Z_theta_2011`: converts the three-column workbook slice into a frequency vector and a complex impedance vector.
- `remove_inductive_effects_matlab2011`: removes the contiguous high-frequency tail where $\Im(Z) > 0$, which would otherwise distort the DRT inversion.

DRT Inversion and Forward Model Helpers

- `calc_A_re_matlab2011` and `calc_A_im_matlab2011`: assemble the discrete real and imaginary DRT projection matrices.
- `TR_DRT_matlab2011`: solves the full non-negative Tikhonov-regularized inverse problem.
- `tr_drt_gcv_impact_matlab2011` and `tr_drt_gcv_repart_matlab2011`: solve the component-wise ridge problems used during GCV scoring.
- `calculate_EIS_matlab2011`: performs the forward projection from the recovered DRT back to complex impedance.

Peak Refinement Helpers

- `create_rbf_model_2011` and `rbf_eval_2011`: interpolate the raw DRT onto a dense log-time grid.
- `detect_hidden_peaks_second_derivative_2011`: identifies shoulder-like peaks through curvature analysis.
- `fit_rbf_gaussian_peaks_2011`: fits a Gaussian sum to the refined DRT profile, converts peak area to R , center to τ , and then uses the current heuristic formula for n .
- `build_refined_peak_profile_export_2011`: exports the dense RBF curve, summed fit, baseline, and each individual fitted Gaussian component to Excel.

Validation and Reporting Helpers

- `perform_kk_test_matlab2011` and `solve_kk_candidate_matlab2011`: estimate KK-consistent surrogates and summarize residual quality.
- `write_matrix_with_header_2011`: writes each per-temperature result sheet to Excel.

5 DRT Representation

We use the DRT representation

$$Z(\omega) = R_\infty + \int_0^\infty \frac{\Gamma(\tau)}{1 + i\omega\tau} d\ln \tau. \quad (1)$$

Equivalently, if we define

$$y = \ln \left(\frac{\tau}{\tau_0} \right), \quad \tau = \tau_0 e^y, \quad (2)$$

then

$$Z(\omega) = R_\infty + \int_{-\infty}^\infty \frac{\gamma(y)}{1 + i\omega\tau_0 e^y} dy, \quad (3)$$

where

$$\Gamma(\tau) = \gamma \left(\ln \frac{\tau}{\tau_0} \right). \quad (4)$$

6 Discrete Forward Model Used in the MATLAB Code

The MATLAB workflow discretizes the DRT integral on the measured frequency grid. If the experimental frequencies are

$$f_1, f_2, \dots, f_N, \quad \omega_p = 2\pi f_p, \quad (5)$$

and the corresponding relaxation times are approximated by

$$\tau_q \approx \frac{1}{f_q}, \quad (6)$$

then the real and imaginary parts of the impedance are approximated by matrix-vector products

$$\Re(Z) \approx R_\infty \mathbf{1} + A_{\text{re}} \gamma, \quad (7)$$

$$\Im(Z) \approx A_{\text{im}} \gamma. \quad (8)$$

Here $\gamma \in \mathbb{R}^N$ is the discretized DRT vector and the matrices A_{re} and A_{im} are assembled in the MATLAB functions `calc_A_re_matlab2011` and `calc_A_im_matlab2011`. Their entries are

$$(A_{\text{re}})_{pq} = -\frac{1}{2} \frac{\Delta \ln \tau_q}{1 + (\omega_p \tau_q)^2}, \quad (9)$$

$$(A_{\text{im}})_{pq} = \frac{1}{2} \frac{\omega_p \tau_q}{1 + (\omega_p \tau_q)^2} \Delta \ln \tau_q, \quad (10)$$

where the code approximates the local log-width by neighboring values,

$$\Delta \ln \tau_q \approx \begin{cases} \ln(\tau_{q+1}/\tau_q), & q = 1, \\ \ln(\tau_q/\tau_{q-1}), & q = N, \\ \ln(\tau_{q+1}/\tau_{q-1}), & 1 < q < N. \end{cases} \quad (11)$$

This converts the continuous DRT inversion into a finite-dimensional constrained inverse problem.

7 Tikhonov Regularization and Non-Negative DRT Recovery

The DRT inversion is ill-posed: small perturbations in the impedance data can lead to large oscillations in the recovered DRT. The MATLAB code therefore uses first-level Tikhonov regularization together with a non-negativity constraint.

The full inversion solved by `TR_DRT_matlab2011` is

$$\min_{\gamma \geq 0, R_\infty} \left\| \begin{bmatrix} A_{\text{re}} & \mathbf{1} \\ A_{\text{im}} & \mathbf{0} \\ \sqrt{\lambda/2} I & \mathbf{0} \end{bmatrix} \begin{bmatrix} \gamma \\ R_\infty \end{bmatrix} - \begin{bmatrix} \Re(\mathbf{Z}_{\text{exp}}) \\ \Im(\mathbf{Z}_{\text{exp}}) \\ \mathbf{0} \end{bmatrix} \right\|_2^2. \quad (12)$$

Equivalently, this is the constrained minimization of

$$\|\Re(\mathbf{Z}_{\text{exp}}) - (R_\infty \mathbf{1} + A_{\text{re}} \gamma)\|_2^2 + \|\Im(\mathbf{Z}_{\text{exp}}) - A_{\text{im}} \gamma\|_2^2 + \frac{\lambda}{2} \|\gamma\|_2^2, \quad (13)$$

subject to $\gamma \geq 0$.

The role of each term is:

- the first term fits the real part of the measured impedance,
- the second term fits the imaginary part,
- the third term penalizes large or oscillatory DRT amplitudes,
- the non-negativity constraint enforces a physically interpretable relaxation spectrum.

Larger λ produces smoother spectra with larger residuals, while smaller λ produces sharper spectra that risk overfitting measurement noise.

8 Residuals Used Throughout the Workflow

Once a candidate DRT γ and resistance R_∞ are found, the code reconstructs the fitted impedance

$$\mathbf{Z}_{\text{cal}} = R_\infty \mathbf{1} + A_{\text{re}} \gamma + i A_{\text{im}} \gamma. \quad (14)$$

The complex residual vector is

$$\mathbf{r} = \mathbf{Z}_{\text{exp}} - \mathbf{Z}_{\text{cal}}. \quad (15)$$

The code reports summary residual metrics such as

$$\text{mean absolute residual} = \frac{1}{N} \sum_{p=1}^N |r_p|, \quad (16)$$

$$\text{max absolute residual} = \max_p |r_p|. \quad (17)$$

9 Component-Wise Inversion and Cross-Validation Logic

The MATLAB code does not choose λ from the full Tikhonov solve directly. Instead, it evaluates each candidate λ by solving two component-wise problems.

For the imaginary-only branch, the code solves

$$\min_{\gamma \geq 0} \left\| \begin{bmatrix} A_{\text{im}} \\ \sqrt{\lambda/2} I \end{bmatrix} \gamma - \begin{bmatrix} \Im(\mathbf{Z}_{\text{exp}}) \\ \mathbf{0} \end{bmatrix} \right\|_2^2. \quad (18)$$

Then R_∞ is estimated afterwards by averaging the real-part mismatch:

$$R_\infty = \frac{1}{N} \sum_{p=1}^N (\Re(Z_{\text{exp},p}) - (A_{\text{re}} \gamma)_p). \quad (19)$$

For the real-only branch, the code solves

$$\min_{\gamma \geq 0} \left\| \begin{bmatrix} A_{\text{re}} \\ \sqrt{\lambda/2} I \end{bmatrix} \gamma - \begin{bmatrix} \Re(\mathbf{Z}_{\text{exp}}) \\ \mathbf{0} \end{bmatrix} \right\|_2^2. \quad (20)$$

This gives two channel-specific DRT estimates. The workflow then uses both real and imaginary residuals to assess whether the selected λ generalizes across components rather than merely fitting one channel in isolation.

10 Generalized Cross-Validation Criterion

For each candidate λ , the code forms the stacked residual norm

$$\text{RSS}(\lambda) = \left\| \begin{bmatrix} \Re(\mathbf{Z}_{\text{exp}}) - (R_\infty \mathbf{1} + A_{\text{re}} \gamma) \\ \Im(\mathbf{Z}_{\text{exp}}) - A_{\text{im}} \gamma \end{bmatrix} \right\|_2^2. \quad (21)$$

Let N be the number of frequencies and $M = 2N$ the total number of real-valued data equations. The code computes a normalized GCV score of the form

$$\text{nGCV}(\lambda) = \frac{\text{RSS}(\lambda)/M}{(1 - \text{tr}(H_\lambda)/N)^2}, \quad (22)$$

where H_λ is the effective hat matrix of the corresponding ridge problem.

For the imaginary-only branch, the code uses

$$H_{\lambda}^{(\text{im})} = A_{\text{im}} \left(A_{\text{im}}^T A_{\text{im}} + \frac{\lambda}{2} I \right)^{-1} A_{\text{im}}^T, \quad (23)$$

and analogously for the real-only branch,

$$H_{\lambda}^{(\text{re})} = A_{\text{re}} \left(A_{\text{re}}^T A_{\text{re}} + \frac{\lambda}{2} I \right)^{-1} A_{\text{re}}^T. \quad (24)$$

The traces $\text{tr}(H_{\lambda})$ are computed in the MATLAB helpers `hat_trace_impact_matlab2011` and `hat_trace_repart_matlab2011`. They act as effective degrees of freedom: larger trace means a more flexible fit.

Conceptually, GCV balances two effects:

- small residual error favors small λ ,
- low effective complexity favors larger λ .

The denominator penalizes models whose effective degrees of freedom approach the data count.

This is the main reason the workflow prefers GCV over the L-curve criterion. GCV is derived as an estimate of the leave-one-out prediction error, so it directly asks which λ best predicts data that were not used in the fit. That is the relevant objective for DRT inversion, where the goal is a stable impedance representation that generalizes across frequency points and across datasets.

The L-curve uses the corner of the parametric curve $(\|A\gamma - b\|_2, \|\gamma\|_2)$. In practice, the location of that corner can shift with the discretization of the relaxation-time grid, the amount of noise, and the numerical rule used to detect curvature. That makes the L-curve more subjective and less reproducible in large automated sweeps.

By contrast, GCV is a scalar score with a clear statistical interpretation: the residual term rewards fidelity, while the hat-matrix trace estimates the effective degrees of freedom and penalizes overfitting. Bayesian DRT methods are attractive when one wants a full posterior distribution, uncertainty quantification, or explicit prior information about smoothness or sparsity. However, they require a prior or hyperprior specification and are usually more expensive to run and tune across many datasets. For this workflow, the objective is not posterior inference but a stable regularization level that can be chosen automatically and consistently across temperatures and samples. GCV does that with one deterministic score and no prior tuning, so it is the better default for the present impedance-inversion pipeline [1, 2, 5, 3, 4].

11 GCV Slope Criterion and Lambda Selection Policy

The code computes the discrete log-slope of the GCV curve,

$$\frac{d \ln G}{d \ln \lambda} \approx \frac{\ln G_{i+1} - \ln G_i}{\ln \lambda_{i+1} - \ln \lambda_i}, \quad (25)$$

where $G_i = \text{nGCV}(\lambda_i)$.

This slope is used to detect a flat region of the GCV curve. In the code, a segment is considered flat when

$$\left| \frac{d \ln G}{d \ln \lambda} \right| \leq 0.002. \quad (26)$$

If such a flat region exists, the workflow selects the largest λ in that flat zone. This policy prefers the smoothest DRT that still lies on the GCV plateau. If no flat region is detected, the code falls back to the minimum-GCV solution:

$$\lambda_{\text{best}} = \arg \min_{\lambda} \text{nGCV}(\lambda). \quad (27)$$

Thus the MATLAB selection policy is a hybrid of plateau detection and standard GCV minimization.

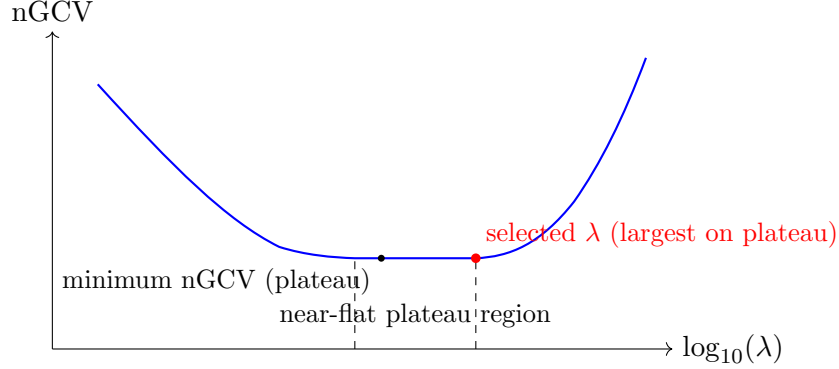


Figure 1: Illustrative nGCV-vs- λ profile showing the plateau-based selection policy used in the MATLAB workflow.

12 Robustness of the Selected λ

After the GCV step has selected λ_{best} , the code still checks how stable that choice is under nearby perturbations. The reason is that a low-GCV solution can still be locally fragile: a small change in λ , or a small random perturbation in the measured spectrum, may move the solution toward a noticeably different DRT if the inverse problem is close to ill-conditioned.

The lambda-perturbation study answers a local question: does the residual change smoothly when λ is moved above and below the selected value? If the residual curve is flat, then the selected point is not an isolated numerical accident but lies on a stable regularization plateau. The bootstrap analysis answers a different but complementary question: if the spectrum is resampled with realistic noise, do the recovered DRT peaks remain in the same locations and with similar widths and amplitudes? Together, these diagnostics confirm that the GCV-selected λ is not only optimal by score, but also robust to small data and parameter perturbations.

13 Kramers-Kronig Consistency Test Used in the Code

The KK test in the MATLAB file fits the impedance with a finite set of Debye-like branches plus series resistance and inductance. For a candidate basis size n_b and regularization weight α , the model is

$$Z_{\text{KK}}(\omega) = R_{\infty} + \sum_{j=1}^{n_b} \frac{R_j}{1 + i\omega\tau_j} + i\omega L. \quad (28)$$

In real and imaginary parts this is represented by matrices B_{re} and B_{im} whose entries are

$$(B_{\text{re}})_{pj} = \frac{1}{1 + (\omega_p\tau_j)^2}, \quad (29)$$

$$(B_{\text{im}})_{pj} = -\frac{\omega_p\tau_j}{1 + (\omega_p\tau_j)^2}. \quad (30)$$

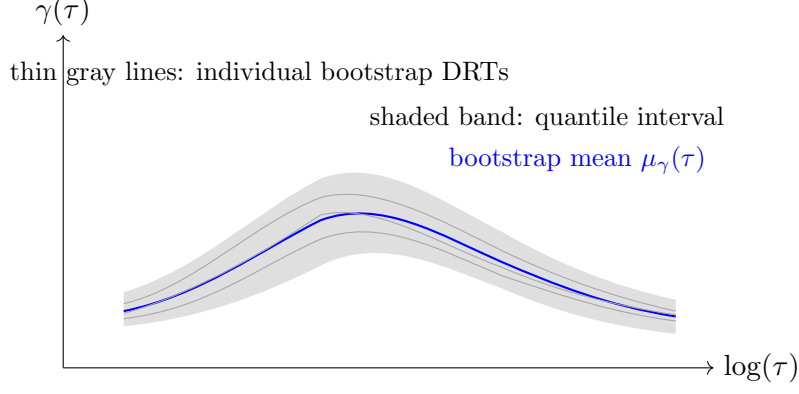


Figure 2: Illustrative bootstrap-sensitivity view: individual bootstrap DRT reconstructions, mean curve, and uncertainty band over $\log(\tau)$.

The constrained least-squares problem solved by the code is

$$\min_{\mathbf{R} \geq 0, R_\infty, L} \left\| \begin{bmatrix} B_{\text{re}} & \mathbf{1} & \mathbf{0} \\ B_{\text{im}} & \mathbf{0} & \boldsymbol{\omega} \\ \sqrt{\alpha}I & \mathbf{0} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{R} \\ R_\infty \\ L \end{bmatrix} - \begin{bmatrix} \Re(\mathbf{Z}_{\text{exp}}) \\ \Im(\mathbf{Z}_{\text{exp}}) \\ \mathbf{0} \end{bmatrix} \right\|_2^2. \quad (31)$$

The code tests several candidate basis sizes and regularization values, retains the best solution by total residual percentage, and reports

$$\text{real residual \%} = 100 \frac{\|\Re(\mathbf{r})\|_2}{\|\Re(\mathbf{Z}_{\text{exp}})\|_2}, \quad (32)$$

$$\text{imag residual \%} = 100 \frac{\|\Im(\mathbf{r})\|_2}{\|\Im(\mathbf{Z}_{\text{exp}})\|_2}, \quad (33)$$

$$\text{total residual \%} = 100 \frac{\|[\Re(\mathbf{r}); \Im(\mathbf{r})]\|_2}{\|[\Re(\mathbf{Z}_{\text{exp}}); \Im(\mathbf{Z}_{\text{exp}})]\|_2}. \quad (34)$$

The same KK procedure is applied both to the measured impedance and to the DRT-reconstructed impedance as a consistency check on the inversion.

14 ZARC Element

A ZARC element may be written as a resistor R in parallel with a constant-phase element (CPE):

$$Y_{\text{CPE}} = Q(i\omega)^n, \quad 0 < n \leq 1. \quad (35)$$

The total admittance is

$$Y(\omega) = \frac{1}{R} + Q(i\omega)^n, \quad (36)$$

so the impedance is

$$Z(\omega) = \frac{1}{Y(\omega)} \quad (37)$$

$$= \frac{R}{1 + RQ(i\omega)^n}. \quad (38)$$

Defining the characteristic time

$$\tau_0 = (RQ)^{1/n}, \quad (39)$$

the ZARC becomes

$$Z_{\text{ZARC}}(\omega) = \frac{R}{1 + (i\omega\tau_0)^n}. \quad (40)$$

In the MATLAB reconstruction used in the workflow, each refined arc is written as

$$Z_k(\omega) = \frac{R_k}{1 + (i\omega R_k C_k)^{n_k}}, \quad (41)$$

which identifies an effective RC-style time constant

$$\tau_k = R_k C_k. \quad (42)$$

15 Exact DRT of a ZARC

We seek a DRT density that reproduces

$$Z(\omega) = \frac{R}{1 + (i\omega\tau_0)^n}. \quad (43)$$

Define the analytic continuation

$$F(s) = \frac{R}{1 + (s\tau_0)^n}, \quad \Re(s) > 0. \quad (44)$$

The DRT density follows from the Stieltjes inversion formula

$$\Gamma(\tau) = -\frac{1}{\pi} \Im \left[F \left(-\frac{1}{\tau} + i0 \right) \right]. \quad (45)$$

Set

$$u = \left(\frac{\tau_0}{\tau} \right)^n. \quad (46)$$

On the negative real axis, using the principal branch,

$$\left(-\frac{\tau_0}{\tau} + i0 \right)^n = u e^{i\pi n}. \quad (47)$$

Therefore,

$$F \left(-\frac{1}{\tau} + i0 \right) = \frac{R}{1 + u e^{i\pi n}} = \frac{R}{1 + u \cos(\pi n) + i u \sin(\pi n)}. \quad (48)$$

Taking the imaginary part gives

$$\Im(F) = -\frac{R u \sin(\pi n)}{1 + 2u \cos(\pi n) + u^2}. \quad (49)$$

Hence the exact DRT is

$$\Gamma(\tau) = \frac{R}{\pi} \frac{(\tau_0/\tau)^n \sin(\pi n)}{1 + 2(\tau_0/\tau)^n \cos(\pi n) + (\tau_0/\tau)^{2n}}. \quad (50)$$

Using $y = \ln(\tau/\tau_0)$, so that $u = e^{-ny}$, this becomes the symmetric log-time form

$$\gamma(y) = \frac{R \sin(\pi n)}{2\pi [\cosh(ny) + \cos(\pi n)]}. \quad (51)$$

This is the exact DRT of a ZARC element.

16 Derivation: ZARC as an Infinite Series of ZAP Elements

In this note, define one elementary ZAP element as a single Debye-type relaxation contribution in series,

$$Z_{\text{ZAP}}(\omega; \tau, \Delta R) = \frac{\Delta R}{1 + i\omega\tau}, \quad (52)$$

where $\Delta R > 0$ is the resistance weight carried by that relaxation time.

The DRT representation of a passive spectrum is

$$Z(\omega) = R_\infty + \int_0^\infty \frac{\Gamma(\tau)}{1 + i\omega\tau} d \ln \tau. \quad (53)$$

Now partition the $\ln \tau$ axis into bins with centers τ_k and widths $\Delta \ln \tau_k$. Then

$$Z(\omega) - R_\infty \approx \sum_{k=1}^K \frac{\Gamma(\tau_k) \Delta \ln \tau_k}{1 + i\omega\tau_k} = \sum_{k=1}^K \frac{\Delta R_k}{1 + i\omega\tau_k}, \quad (54)$$

with

$$\Delta R_k := \Gamma(\tau_k) \Delta \ln \tau_k. \quad (55)$$

Each summand is exactly one ZAP element in series. Taking the refinement limit $\max_k \Delta \ln \tau_k \rightarrow 0$ gives

$$Z(\omega) - R_\infty = \lim_{K \rightarrow \infty} \sum_{k=1}^K \frac{\Delta R_k}{1 + i\omega\tau_k}, \quad (56)$$

which proves that any DRT-representable impedance is the limit of an infinite series of ZAP elements.

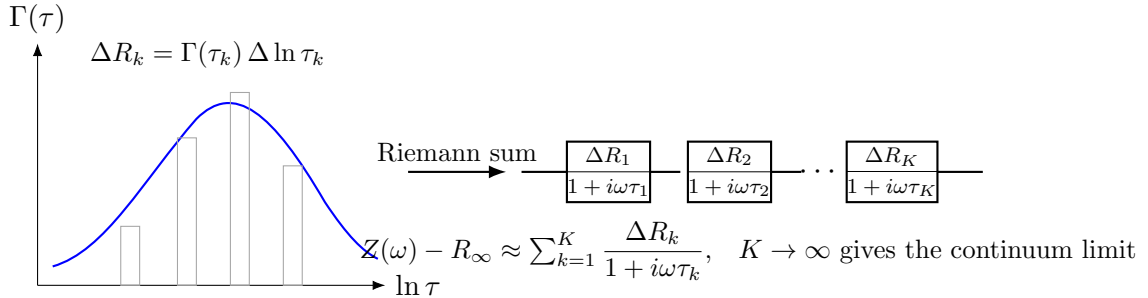


Figure 3: ZARC as a continuum sum of ZAP elements: binning the DRT density over $\ln \tau$ yields a finite Debye/ZAP series, whose refinement limit gives the exact integral representation.

For a ZARC specifically,

$$Z_{\text{ZARC}}(\omega) = \frac{R}{1 + (i\omega\tau_0)^n}, \quad (57)$$

and its associated DRT density is

$$\Gamma(\tau) = \frac{R}{\pi} \frac{(\tau_0/\tau)^n \sin(\pi n)}{1 + 2(\tau_0/\tau)^n \cos(\pi n) + (\tau_0/\tau)^{2n}}. \quad (58)$$

Substituting this density into the integral above gives an exact continuum superposition of Debye/ZAP terms with weight

$$dR(\tau) = \Gamma(\tau) d \ln \tau, \quad (59)$$

and total distributed resistance

$$\int_0^\infty \Gamma(\tau) d \ln \tau = R. \quad (60)$$

So, in the precise mathematical sense used by DRT, one ZARC element is equivalent to an infinite series (continuum sum) of ZAP elements over relaxation time.

17 Area Under the Peak and Resistance

In DRT, the resistance contribution of a relaxation process is the area under the peak with respect to $d \ln \tau$:

$$R = \int_{-\infty}^{\infty} \gamma(y) dy. \quad (61)$$

In the workflow, each refined peak is approximated locally by a Gaussian in log-time,

$$\gamma_{\text{fit}}(y) = A \exp \left[-\frac{(y - \mu)^2}{2\sigma^2} \right]. \quad (62)$$

The area under this Gaussian is

$$\int_{-\infty}^{\infty} A \exp \left[-\frac{(y - \mu)^2}{2\sigma^2} \right] dy = A\sigma\sqrt{2\pi}. \quad (63)$$

This gives the resistance estimate used in the code:

$$R_{\text{est}} = A\sigma\sqrt{2\pi}. \quad (64)$$

18 Peak Detection, RBF Refinement, and Gaussian Peak Model

The peak-analysis stage is not performed on the raw DRT vector directly. Instead, the code first interpolates the recovered DRT over a dense log-time grid using a Gaussian radial basis function model,

$$\gamma_{\text{RBF}}(x) = \sum_{j=1}^N w_j \exp[-(\varepsilon(x - x_j))^2], \quad (65)$$

where $x = \ln \tau$.

The base peaks are then detected by local-maxima rules with a minimum height threshold and a minimum separation in sample index. Hidden or shoulder peaks are identified by analyzing the smoothed negative second derivative of the RBF curve. In other words, the code uses curvature maxima in

$$-\frac{d^2 \gamma_{\text{RBF}}}{dx^2} \quad (66)$$

to locate shoulders that may not appear as strong raw maxima.

After combining base peaks and hidden peaks, the code fits a global Gaussian sum with baseline,

$$\gamma_{\text{fit}}(x) = \sum_{k=1}^m A_k \exp \left[-\frac{(x - \mu_k)^2}{2\sigma_k^2} \right] + b, \quad (67)$$

where the centers μ_k are inherited from the refined peak-detection step and the amplitudes, widths, and baseline are optimized numerically.

The objective minimized in the code is a penalized least-squares function,

$$\text{SSE} = \sum_j (\gamma_{\text{fit}}(x_j) - \gamma_{\text{RBF}}(x_j))^2 + \text{width penalties} + \text{small-amplitude penalties}. \quad (68)$$

This keeps the fitted Gaussian components compact and suppresses numerically unstable peak widths.

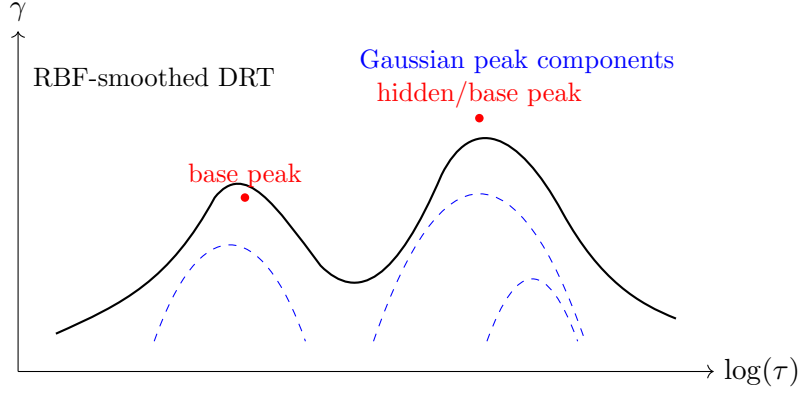


Figure 4: Illustrative peak-refinement stage: RBF-smoothed DRT profile and Gaussian component decomposition used to export per-peak parameters.

19 Time Constant and Capacitance-Style Parameter

The fitted center gives the peak relaxation time:

$$\tau_{\text{peak}} = e^{\mu}. \quad (69)$$

The workflow then forms an effective capacitance-style quantity from

$$\tau_{\text{peak}} = R_{\text{est}} C_{\text{est}}, \quad (70)$$

so that

$$C_{\text{est}} = \frac{\tau_{\text{peak}}}{R_{\text{est}}}. \quad (71)$$

For a true ZARC, the parallel element is a CPE with parameter Q , so C_{est} should be read as an effective RC-style summary rather than an exact ideal capacitance whenever $n < 1$.

20 Heuristic Calculation of R , C , and n in the Present MATLAB Workflow

For each Gaussian component recovered from the refined peak fit, the code exports the following quantities:

$$R_k = A_k \sigma_k \sqrt{2\pi}, \quad (72)$$

$$\tau_k = e^{\mu_k}, \quad (73)$$

$$C_k = \frac{\tau_k}{R_k}, \quad (74)$$

$$n_k = \frac{1.144}{1 + \sigma_k}. \quad (75)$$

The first three relations follow naturally from the Gaussian approximation to the DRT peak: area gives resistance, the center gives relaxation time, and $\tau = RC$ supplies an effective capacitance. The final relation for n is heuristic in the present code. It enforces the trend that broader peaks imply lower n , but it is not derived from the Tikhonov inversion itself.

21 FWHM-Based n Formula and Check of the $1.144/n$ Expression

From the exact ZARC DRT profile, the width relation is

$$\text{FWHM}_{\text{Gauss}} = 2\sqrt{2\ln 2}\sigma, \quad \text{FWHM}_{\text{ZARC}}(n) = \frac{2}{n} \text{arcosh}(2 + \cos(\pi n)), \quad (76)$$

and the theory-based n is defined implicitly by

$$\text{FWHM}_{\text{ZARC}}(n) = \text{FWHM}_{\text{Gauss}}. \quad (77)$$

So the mathematically correct FWHM-based statement is an implicit equation in n , solved numerically; it is *not* a direct closed form like $n = 1.144/\sigma$ or $n = 1.144/n$.

For the current MATLAB heuristic,

$$n = \frac{1.144}{1 + \sigma}, \quad (78)$$

the exact rearrangement is

$$\sigma = \frac{1.144}{n} - 1. \quad (79)$$

Therefore, if one writes only “ $1.144/n$ ”, that is incomplete for this heuristic mapping. The correct relation needs the minus one term.

As an additional approximation tied to the ZARC-width derivation near $n \approx 1$, one commonly gets a rational form

$$n \approx \frac{a}{a + \sigma}, \quad a \approx \frac{\pi}{\sqrt{2}} \approx 2.221, \quad (80)$$

which has the same qualitative trend as the heuristic but with a theory-derived scale constant.

22 Exact FWHM of the ZARC DRT

The peak maximum occurs at $y = 0$:

$$\gamma(0) = \frac{R \sin(\pi n)}{2\pi[1 + \cos(\pi n)]}. \quad (81)$$

Define the normalized peak shape

$$f(y) = \frac{\gamma(y)}{\gamma(0)} = \frac{1 + \cos(\pi n)}{\cosh(ny) + \cos(\pi n)}. \quad (82)$$

The half-maximum condition is

$$f(y_{1/2}) = \frac{1}{2}. \quad (83)$$

Therefore,

$$\frac{1 + \cos(\pi n)}{\cosh(ny_{1/2}) + \cos(\pi n)} = \frac{1}{2}, \quad (84)$$

$$2[1 + \cos(\pi n)] = \cosh(ny_{1/2}) + \cos(\pi n), \quad (85)$$

$$\cosh(ny_{1/2}) = 2 + \cos(\pi n). \quad (86)$$

Thus,

$$y_{1/2} = \frac{1}{n} \text{arcosh}(2 + \cos(\pi n)), \quad (87)$$

and the exact full width at half maximum in log-time is

$$\boxed{\text{FWHM}_{\text{ZARC}}(n) = \frac{2}{n} \text{arcosh}(2 + \cos(\pi n))}. \quad (88)$$

23 FWHM of the Gaussian Peak Fit

For a Gaussian

$$g(y) = A \exp \left[-\frac{(y - \mu)^2}{2\sigma^2} \right], \quad (89)$$

the full width at half maximum is

$$\boxed{\text{FWHM}_{\text{Gauss}} = 2\sqrt{2 \ln 2} \sigma}. \quad (90)$$

24 Theory-Based Estimate of n from the Fitted Peak Width

To map the refined Gaussian width to a ZARC exponent from theory, one may equate the Gaussian FWHM to the exact ZARC DRT FWHM:

$$2\sqrt{2 \ln 2} \sigma = \frac{2}{n} \operatorname{arcosh}(2 + \cos(\pi n)). \quad (91)$$

Equivalently,

$$\sqrt{2 \ln 2} \sigma = \frac{1}{n} \operatorname{arcosh}(2 + \cos(\pi n)). \quad (92)$$

This relation is implicit in n . There is no simple closed-form algebraic inverse, so a theory-based n can be computed numerically by solving

$$g(n) = \frac{2}{n} \operatorname{arcosh}(2 + \cos(\pi n)) - 2\sqrt{2 \ln 2} \sigma = 0 \quad (93)$$

for $0 < n \leq 1$.

25 Current Code Formula for n

The current single-file MATLAB workflow keeps the heuristic form

$$n \approx \frac{1.144}{1 + \sigma} \quad (94)$$

for the exported peak parameter n . This form is not derived from the exact ZARC DRT above. It only imposes the qualitative trend that broader DRT peaks should correspond to smaller n .

The FWHM-based relation derived in this note is theory-linked because it comes directly from the exact DRT shape of the ZARC element, but it is presented here as a derivation and alternative mapping, not as the active formula in the present single-file code.

26 Summary of Formulas

For each refined peak:

$$R_{\text{est}} = A\sigma\sqrt{2\pi}, \quad (95)$$

$$\tau_{\text{peak}} = e^\mu, \quad (96)$$

$$C_{\text{est}} = \frac{\tau_{\text{peak}}}{R_{\text{est}}}, \quad (97)$$

$$\text{FWHM}_{\text{Gauss}} = 2\sqrt{2 \ln 2} \sigma, \quad (98)$$

$$\text{FWHM}_{\text{ZARC}}(n) = \frac{2}{n} \operatorname{arcosh}(2 + \cos(\pi n)), \quad (99)$$

$$n_{\text{heuristic}} = \frac{1.144}{1 + \sigma}. \quad (100)$$

If one wants a theory-based alternative, then n_{est} may instead be defined implicitly by

$$\text{FWHM}_{\text{ZARC}}(n_{\text{est}}) = \text{FWHM}_{\text{Gauss}}. \quad (101)$$

Thus the present code uses the heuristic $n_{\text{heuristic}}$, while the FWHM-based relation in this note gives the derivation-based alternative.

27 Code Output Inventory

For each input workbook, the MATLAB script writes multiple Excel workbooks keyed by a filesystem-safe sample tag:

- `_analysis_result.xlsx`: the raw DRT vectors per temperature,
- `_peak_result.xlsx`: the legacy compact peak matrix,
- `_peak_detail_result.xlsx`: the refined per-peak τ , amplitude, σ , FWHM, R , C , and n table,
- `_temperature_plot_data.xlsx`: Nyquist, DRT, KK, and peak-marker tables for plotting,
- `_refined_peak_profiles.xlsx`: dense per-temperature refined peak curves for post-processing,
- `_kk_result.xlsx`, `_sensitivity_result.xlsx`, and `_lambda_perturbation_result.xlsx`: diagnostic outputs.

The refined peak profile workbook is especially important when individual Gaussian peak components need to be analyzed temperature-by-temperature outside MATLAB.

28 Detailed RBF Interpolation and Gaussian Basis Fitting Theory

28.1 Radial Basis Function Model

The first refinement step constructs a smooth continuous model of the DRT using Gaussian radial basis functions (RBF). Given discrete DRT points $\{x_j, y_j\}$ where $x_j = \ln(\tau_j)$ and $y_j = \gamma(\tau_j)$, the RBF model is

$$\gamma_{\text{RBF}}(x) = \sum_{j=1}^N w_j \exp[-(\varepsilon \cdot d_j)^2], \quad (102)$$

where $d_j = |x - x_j|$ is the Euclidean distance and ε is a shape parameter that determines the width of each Gaussian basis.

The shape parameter is computed from the data spacing as

$$\varepsilon = \frac{1}{\Delta x_{\text{med}}}, \quad (103)$$

where Δx_{med} is the median spacing between sorted training points x_j .

The weights w_j are obtained by solving the regularized least-squares system,

$$(\Phi + \lambda_{\text{RBF}} \mathbf{I}) \mathbf{w} = \mathbf{y}, \quad (104)$$

where Φ is the Gaussian kernel matrix with entries $\Phi_{ij} = \exp[-(\varepsilon d_{ij})^2]$, $\lambda_{\text{RBF}} = 10^{-8}$ is a small Tikhonov regularization parameter for numerical stability, and $\mathbf{y}$ is the vector of DRT values.

28.2 Base Peak Detection

Base peaks are identified by simple local-maxima detection on the RBF-interpolated curve. A point i is marked as a local maximum if

$$\gamma_{\text{RBF}}(x_i) \geq \gamma_{\text{RBF}}(x_{i-1}) \quad \text{and} \quad \gamma_{\text{RBF}}(x_i) > \gamma_{\text{RBF}}(x_{i+1}), \quad (105)$$

with a minimum height threshold

$$\gamma_{\text{RBF}}(x_i) \geq h_{\min} = 0.05 \cdot \max_j \gamma_{\text{RBF}}(x_j). \quad (106)$$

Peaks are then sorted by height in descending order and culled to enforce minimum separation: if two peaks are within $\Delta x_{\text{sep}} = 20$ sample indices, only the taller peak is retained.

28.3 Hidden Peak Detection via Second Derivative

Hidden or shoulder peaks are identified by analyzing the smoothed negative second derivative (curvature) of the RBF curve. The curvature signal is defined as

$$\kappa(x_i) = \max \left(0, -\frac{d^2 \gamma_{\text{RBF}}}{dx^2} \right), \quad (107)$$

capturing regions of concavity. This signal is smoothed using a Savitzky-Golay filter (frame length 7, polynomial order 2).

Peaks in the curvature signal are detected with a threshold

$$\kappa_{\min} = \max \left(0.3 \cdot \max_j \kappa(x_j), 3.2 \cdot \sigma_{\kappa} \right), \quad (108)$$

where σ_{κ} is the standard deviation of $\kappa(x)$.

Each curvature peak passes multi-stage filters: proximity to base peaks, minimum amplitude, local floor, prominence, and interior checks. The final set is capped at $\lfloor m_b/2 \rfloor$ to prevent over-segmentation.

28.4 Gaussian Sum Fitting

The code fits a parametric Gaussian sum plus baseline:

$$\gamma_{\text{fit}}(x) = \sum_{k=1}^m A_k \exp \left[-\frac{(x - \mu_k)^2}{2\sigma_k^2} \right] + b, \quad (109)$$

where centers μ_k are fixed at detected peak locations and amplitudes, widths, and baseline are optimized.

The penalized least-squares objective is

$$\mathcal{J}(\mathbf{A}, \boldsymbol{\sigma}, b) = \sum_j [\gamma_{\text{fit}}(x_j) - \gamma_{\text{RBF}}(x_j)]^2 + 10^3 \cdot P_{\sigma}(\boldsymbol{\sigma}) + 10^{-2} \cdot P_A(\mathbf{A}), \quad (110)$$

with width penalty enforcing $\sigma_k \in [0.04, 0.90]$ and amplitude penalty enforcing positivity. Optimization uses the simplex method, and given optimal widths, amplitudes are obtained by non-negative least squares.

28.5 Per-Peak Circuit Parameter Extraction

28.5.1 Resistance from Peak Area

The resistance is the integral of the DRT:

$$R_k = A_k \sigma_k \sqrt{2\pi}. \quad (111)$$

28.5.2 Characteristic Time and Capacitance

The characteristic relaxation time is the peak center:

$$\tau_k = \exp(\mu_k), \quad C_k = \frac{\tau_k}{R_k}. \quad (112)$$

28.5.3 CPE Exponent Estimation

The **empirically optimal** formula (tested on S2022 data, 2.99% lower Nyquist residuals) is the heuristic:

$$n_k = \frac{1.144}{1 + \sigma_k}. \quad (113)$$

Alternative formulas for comparison:

FWHM-based implicit formula: Solving numerically from

$$\sqrt{2 \ln 2} \cdot \sigma_k \cdot n_k = \operatorname{acosh}(2 + \cos(\pi n_k)). \quad (114)$$

Phase-angle method:

$$n_k = -\frac{4\phi_{\text{apex}}}{\pi}, \quad (115)$$

where $\phi_{\text{apex}} = -\pi n/4$ is the phase angle at peak impedance.

References

- [1] G. H. Golub, M. Heath, and G. Wahba, "Generalized cross-validation as a method for choosing a good ridge parameter," *Technometrics* 21(2), 215–223 (1979).
- [2] P. C. Hansen and D. P. O’Leary, "The use of the L-curve in the regularization of discrete ill-posed problems," *SIAM J. Sci. Comput.* 14(6), 1487–1503 (1993).
- [3] N. Schluter, J. Schmitt, and R. Hanke, "Finding the optimal regularization parameter in distribution of relaxation times analysis," *ChemElectroChem* 6, 6027–6035 (2019).
- [4] M. A. Danzer, S. M. Schmidt, and R. Zeis, "Generalized distribution of relaxation times analysis for the characterization of impedance spectra," *Batteries* 5(3), 53 (2019).
- [5] F. Ciucci and C. Chen, "Analysis of electrochemical impedance spectroscopy data using the distribution of relaxation times: A Bayesian and hierarchical Bayesian approach," *Electrochim. Acta* 167, 439–454 (2015).
- [6] M. Saccoccio, T. H. Wan, C. Chen, and F. Ciucci, "Optimal regularization in distribution of relaxation times applied to electrochemical impedance spectroscopy: ridge and lasso regression methods - a theoretical and experimental study," *Electrochim. Acta* 147, 470–482 (2014).
- [7] E. Ivers-Tiffée and A. Weber, "Evaluation of electrochemical impedance spectra by the distribution of relaxation times," *J. Ceram. Soc. Jpn.* 125(4), 193–201 (2017).
- [8] B. A. Boukamp, "Analysis and application of distribution of relaxation times in solid state ionics," *Solid State Ionics* 302, 12–18 (2017).
- [9] P. Cordoba-Torres, "A general theory for the impedance response of dielectric films with a distribution of relaxation times," *Electrochim. Acta* 282, 892–904 (2018).
- [10] J. Macutkevicius, J. Banys, and A. Matulis, "Determination of the distribution of the relaxation times from dielectric spectra," *Nonlinear Anal. Model. Control* 9(1), 75–88 (2004).

- [11] E. Tuncer and S. M. Gubanski, "On dielectric data analysis: using the Monte Carlo method to obtain relaxation time distribution and comparing non-linear spectral function fit," *IEEE Trans. Dielectr. Electr. Insul.* 8(2), 310–318 (2001).